

DAA

- o Computational process to solve a problem.
- o The term "Analysis" of algorithm coined by Donald Knuth
- o Analysis is the process of comparing two algorithms w.r.t time, space etc.

Types of Analysis

There are two types of analysis:

Priori

- Done before the execution of an algorithm.
- It is independent from hardware and programming language.
- Approximate analysis is done
 - o Time Complexity
 - o Space Complexity

Posteriori

- Done after the execution of an algorithm.
- It is dependent on system configuration and varies from system to system and depends on programming language, so it is not much effective.

- it depends on the no of statements are executed.	- Generally in industry they don't a posteriori analysis.
- it is done using asymptotic notations.	
- it will give approximate answers.	- it will give exact answers.
- if the algorithm running faster, credit goes to the programmer.	- if the time taken by the program is less then the credit will go to compiler & hardware.

Q. Why do analyze algorithm?

- To solve a problem there are multiple algorithms we need to choose one algorithm that is best for us.
- for example to find shortest path there are multiple algorithms.

Q. How to analyze algorithm?

Using two factors

- Time Complexity
- Space Complexity

Asymptotic notations: / Growth of functions

- The notations used to show the behaviour of function $f(n)$, when value of the argument (typically n) approaches to infinite.
- These are the mathematical notations used to describe the running time of an algorithm when the input tends towards a particular value or a limiting value.

example:

$\Rightarrow$ When the input array is already sorted, the time taken by the algorithm is linear.

$\Rightarrow$ But, when the input array is in reverse condition, the algorithm takes the maximum time (quadratic) to sort the elements.

$\Rightarrow$ When the input array is neither sorted nor in reverse order, then it takes average time.

Types of Asymptotic notation

o These are three types of asymptotic notations:

1. Big Oh O

2. Big Omega Ω

3. Big Theta Θ

Big Oh:

o it is used to represent the upper bound of the running time of an algorithm.

o it measures the worst case time complexity or the maximum amount of time an algorithm can possibly take to complete.

$\Rightarrow$ A function $f(n)$ is said to be in $O(g(n))$ such that

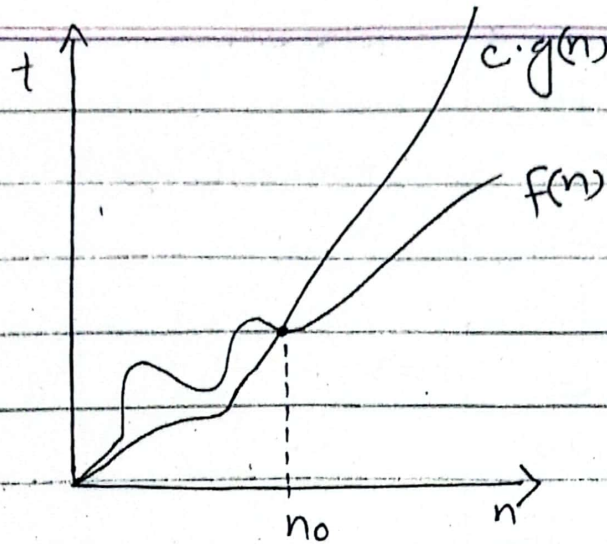
$$f(n) \in O(g(n)) \text{ if } 0 \leq f(n) \leq c g(n)$$

where c and n_0 are +ve

constant i.e. $n_0 > 0$ and $c > 0$

Graphical representation:

$n \geq n_0$



Examples:

$$f(n) = 2n^2 + 5n + 1$$

$$g(n) = n^2, \quad c = 8$$

$$\therefore f(n) \leq c(g(n))$$

$$2n^2 + 5n + 1 \leq 8n^2$$

n	f(n)	c(g(n))
0	1	0
1	8	8
2	19	16
3	34	72

$$f(n) = O(n^2)$$

$$\forall n \geq 1$$

Example 2:

$$f(n) = n + 3$$

$$g(n) = n$$

$$c = 2$$

$$0 \leq n + 3 \leq 2n$$

n	f(n)	c(g(n))	
0	3	0	$f(n) = O(n) \forall n \geq 3$
1	4	2	
2	5	4	
3	6	6	
4	7	8	
5	8	16	

-no=3

Big Omega

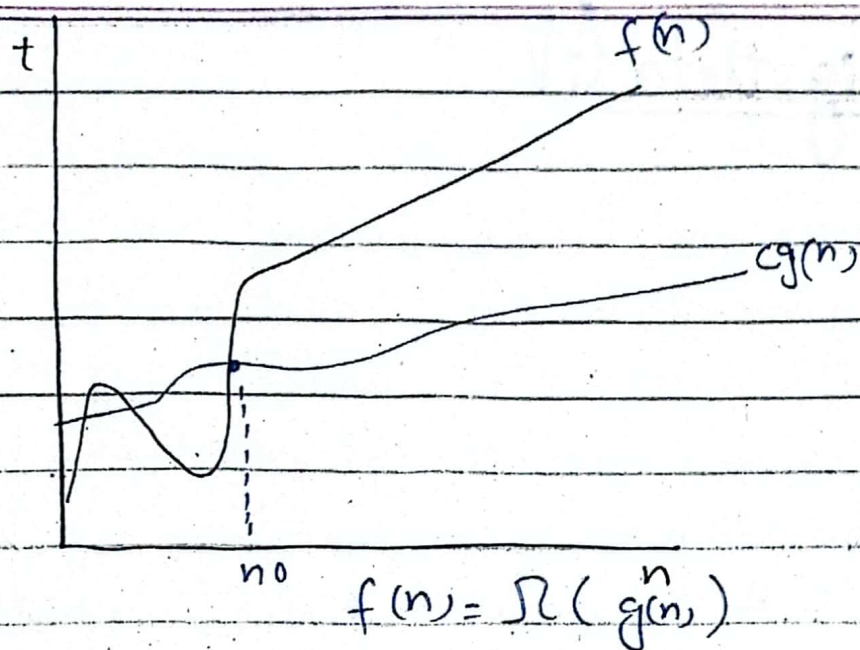
$\Rightarrow$ It is used to represent lower bound of the running time of an algorithm.

$\Rightarrow$ It measures the (lower bound) the best case time complexity or the minimum amount of time an algorithm can possibly take to complete.

A function is said to be in $\Omega(g(n))$ such that $f(n) \in \Omega(g(n))$ if

$$0 < c g(n) \leq f(n) \\ n \geq n_0 \\ c > 0 \\ n_0 > 0$$

$$0 \leq c g(n) \leq f(n)$$



$\Omega(g(n)) = \{ f(n) : \text{there exist +ive constant } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n > n_0 \}$

Example:

$$f(n) = 2n^2 + 3n + 5$$

$$g(n) = n^2, \quad c = 2$$

n	$f(n)$	$c(g(n))$
1	10	2
2	19	8
3	32	18
4	43	32

$$f(n) = \Omega(g(n)) \quad \forall n \geq 1$$

Big theta Θ

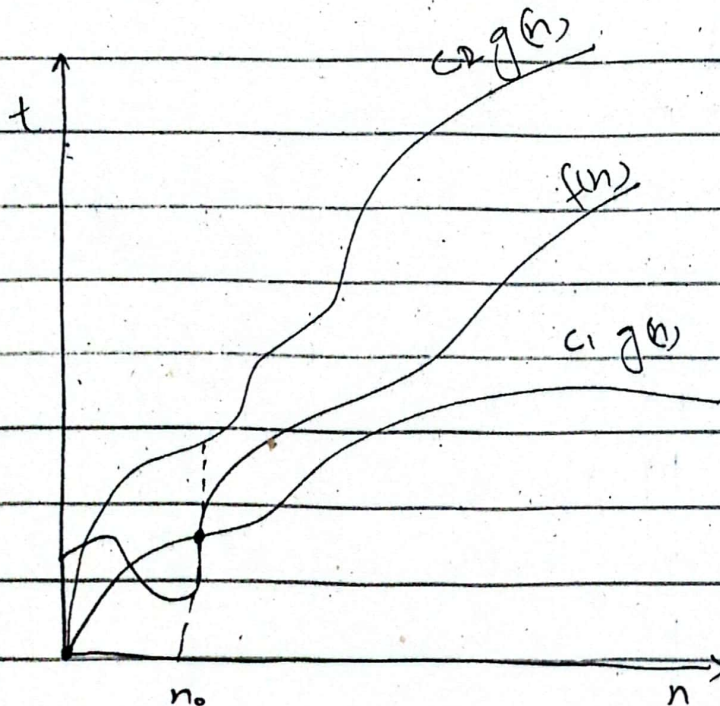
$\Rightarrow$ it is used to represent the upper and lower bound of the running time of an algorithm.

$\Rightarrow$ it is used to analyzing the average-case complexity of an algorithm.

$\Rightarrow$ A function $f(n)$ is said to be in $\Theta(g(n))$ such that

$$f(n) \in \Theta(g(n)) \text{ if } c_1(g(n)) \leq f(n) \leq c_2(g(n))$$
$$n > n_0$$

$$c_1 > 0, c_2 > 0$$



Example:

$$f(n) = 2n + 3$$

$$g(n) = n$$

$$c_1 = 1$$

$$c_2 = 4$$

n	$c_1 g(n)$	$f(n)$	$c_2 g(n)$
1	1	5	4
2	2	7	8
3	3	9	12
4	4	11	16
5	5	13	20
6	6	15	24
7	7	17	28
8	8	19	32

Little Oh:

$\Rightarrow$ it is used to represent upper bound that is not tight or loose upper bound of $f(n)$.

$\Rightarrow$ A function $f(n)$ is said to be $o(g(n))$ such that $f(n) \in o(g(n))$ if

$$0 \leq f(n) < c g(n)$$

where n and c are time constant

$$n \geq n_0$$

Little Omega (ω)

$\Rightarrow$ it is used to represent loose lower bound that is not tightly lower bound.

it is denoted by

$\Rightarrow$ A function $f(n)$ is said to be $\omega(g(n))$ such that $f(n) \in \omega(g(n))$.

if

$$0 \leq c(g(n)) < f(n)$$

$\Rightarrow$ it is denoted by ω .

Expressing Asymptotic notation using Limit theorem

Let $f(n)$ and $g(n)$ be functions.
then

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \quad \text{then} \quad f(n) = o(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \quad \text{then} \quad f(n) = \omega(g(n))$$

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c, \quad \text{then} \quad f(n) = \Theta(g(n)) \quad \text{when } c > 0$$

Example

Prove that $2n^2 + 5 = O(n^3)$

$$= \lim_{n \rightarrow \infty} \frac{2n^2 + 5}{n^3}$$

$$= \lim_{n \rightarrow \infty} \left(\frac{2n^2}{n^3} + \frac{5}{n^3} \right)$$

$$= \lim_{n \rightarrow \infty} \left(\frac{2}{n} + \frac{5}{n^3} \right)$$

$$= \lim_{n \rightarrow \infty} \left(\frac{2}{n} \right) + \lim_{n \rightarrow \infty} \left(\frac{5}{n^3} \right)$$

$$= 0 + 0 = 0$$

Hence proved that

Example:

Prove that $n^2 + 5 = \Theta(n^2)$

$$= \lim_{n \rightarrow \infty} \frac{n^2 + 5}{n^2}$$

$$= \lim_{n \rightarrow \infty} \frac{n^2}{n^2} + \frac{5}{n^2}$$

$$= \lim_{n \rightarrow \infty} \left(1 + \frac{5}{n^2} \right)$$

$$= 1 + \lim_{n \rightarrow \infty} \left(\frac{5}{n^2} \right)$$

$$= 1 + \frac{5}{\infty}$$

$$= 1 + 0$$

$$= 1 \quad \text{Hence proved}$$

Show that $2^{n+1} = \Theta(2^n)$.

$$\begin{aligned} & \lim_{n \rightarrow \infty} \frac{2^{n+1}}{2^n} \\ &= \lim_{n \rightarrow \infty} 2^{n+1-n} \\ &= \lim_{n \rightarrow \infty} 2 \\ &= 2 \end{aligned}$$

L' Hospital rule:

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}$$

Show that $\log(n) = O(n)$.

$$= \lim_{n \rightarrow \infty} \frac{d}{dn} \log(n)$$

$$\frac{d}{dn} (n)$$

$$= \lim_{n \rightarrow \infty} \frac{1}{n}$$

$$= \lim_{n \rightarrow \infty} \frac{1}{n} = \frac{1}{\infty} = 0$$

Show that $3n^2 + 2n + 1 = \Theta(n^2)$

$$f(n) = 3n^2 + 2n + 1$$

$$g(n) = n^2$$

$$f(n) \leq c_1 g(n)$$

$$3n^2 + 2n + 1 \leq n^2$$

$$\div b/s \ n^2$$

$$\Rightarrow 3 + \frac{2}{n} + \frac{1}{n^2} \leq C_1 \quad n=1$$

$$\Rightarrow 3 + 2 + 1 \leq C_1$$

$$6 \leq 6$$

$$3n^2 + 2n + 1 \leq 6n^2$$

$$3 + 2 + 1 \leq 6n^2$$

$$6 \leq 6$$

$$C_1 = 6, \ n_0 = 1$$

$$f(n) \geq C_2 g(n)$$

$$3n^2 + 2n + 1 \geq C_2 n^2$$

$$6 \geq C_2$$

$$C_2 = 1$$

$$3n^2 + 2n + 1 \geq n^2$$

$$3 + 2 + 1 \geq 1$$

$$6 \geq 1$$

$$C_1 = 6, \ C_2 = 1, \ n_0 \geq 1$$

Prove that $3n+2 = \Omega(n)$

$$Cg(n) \leq f(n)$$

$$cn \leq 3n+2$$

$$cn - 3n \leq 2$$

$$n(c-3) \leq 2$$

$$n \leq \frac{2}{c-3}$$

if we assume $c=4, \ n_0=2$

write list of common asymptotic notations.

constant

$$O(1)$$

logarithmic

$$O(\log n)$$

linear

$$O(n \log n)$$

$n \log n$

$$O(n)$$

quadratic

$$O(n^2)$$

cubic

$$O(n^3)$$

polynomial

$$n^{O(1)}$$

exponential

$$2^{O(1)}$$

$$2^n$$

Prove that

$$\frac{1}{2}n^2 - \frac{1}{n}n = \Theta(n^2)$$

$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

$$c_1 n^2 \leq \frac{1}{2}n^2 - \frac{1}{n}n \leq c_2 n^2$$

$$c_1 n^2 \leq \frac{1}{2}n^2 - \frac{1}{n}n$$

$$\therefore c_1 = \frac{1}{4}$$

$$\frac{1}{4}n^2 \leq \frac{1}{2}n^2 - \frac{1}{n}n$$

$$n_0 \geq 2$$

$$c = \frac{1}{4}$$

$$\frac{1}{2}n^2 - \frac{1}{n}n \leq c_2 n^2$$

$$\therefore c_2 = \frac{1}{2}$$

$$\frac{1}{2}n^2 - \frac{1}{n}n \leq \frac{1}{2}n^2$$

$$n_0 \geq 1$$

$$c = \frac{1}{2}, n \geq 1$$

Show that $\frac{1}{2}n^2 - 3n = \Theta(n^2)$.

$$c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$$

$\div$ by n^2

$$c_1 \leq \frac{1}{2} - \frac{3}{n} \leq c_2$$

$$c_1 \leq \frac{1}{2} - \frac{3}{n}$$

$$\frac{1}{2} - \frac{3}{n} \leq c_2$$

$$\therefore c_1 = \frac{1}{14}$$

$$\therefore n_0 = 7$$

$$\frac{1}{14} \leq \frac{1}{2} - \frac{3}{7}$$

$$c = \frac{1}{2}$$

$$\frac{1}{14} \leq \frac{7-21}{14}$$

$$\frac{1}{2} - \frac{3}{7} \leq \frac{1}{2}$$

$$\frac{7-6}{14} \leq \frac{1}{2}$$

$$\frac{1}{14} \leq 1$$

$$\frac{1}{14} \leq \frac{1}{2}$$

$$n > 7$$

Important formulas

$$\sum_{i=l}^u c a_i = c \sum_{i=l}^u a_i, \quad \sum_{i=l}^u i^2 = \frac{n(n+1)(2n+1)}{6}$$

$$\sum_{i=l}^u (a_i + b_i) = \sum_{i=l}^u a_i + \sum_{i=l}^u b_i$$

$$\sum_{i=l}^u 1 = (u - l + 1), \quad \sum_{i=l}^u i^3 = \left(\frac{n(n+1)}{2} \right)^2$$

$$\sum_{i=0}^u i = 1 + 2 + 3 + 4 + \dots + n = \frac{n(n+1)}{2}$$

Finding running time of Bubble sort

BubbleSort($A[0 \dots n-1]$)

$$\sum_{i=0}^{n-2} \left\{ \begin{array}{l} \text{for } i = 0 \text{ to } n-2 \\ \quad \left\{ \begin{array}{l} \text{for } j = 0 \text{ to } n-i-1 \\ \quad \left\{ \begin{array}{l} \text{if } A[j] > A[j+1] \\ \quad \text{Swap } A[j] \text{ \& } A[j+1] \end{array} \right. \end{array} \right. \end{array} \right.$$

$$= \sum_{i=0}^{n-2} \left(\sum_{j=0}^{n-2-i} (1) \right)$$

$$= \sum_{i=0}^{n-2} (n-2-i-0+1)$$

$$= \sum_{i=0}^{n-2} (n-i-1)$$

$$\begin{aligned}
&= \sum_{i=0}^{n-2} n - \sum_{i=0}^{n-2} i - \sum_{i=0}^{n-2} 1 \\
&= n \sum_{i=0}^{n-2} 1 - \sum_{i=0}^{n-2} i - \sum_{i=0}^{n-2} 1 \\
&= n(n-2-0+1) - \frac{(n-2)(n-1)}{2} - (n-2-0+1) \\
&= n(n-1) - \frac{(n-2)(n-1)}{2} - (n-1) \\
&= (n-1) \left(n - \frac{(n-2)}{2} - 1 \right) \\
&= (n-1) \left[\frac{2n - n + 2 - 2}{2} \right] \\
&= (n-1) \left[\frac{n}{2} \right] = \frac{n(n-1)}{2} \\
&= \frac{n^2}{2} - \frac{n}{2} \quad \therefore c = \frac{1}{2} \\
&= cn^2 - cn \\
&\in O(n^2)
\end{aligned}$$

$$\begin{aligned}
\therefore \sum_{i=0}^n 2i &= n(n+1) \\
\therefore \sum_{i=0}^n \frac{1}{2^i} &= 2 - \frac{1}{2^{n-1}} \\
\therefore \sum_{i=1}^n 2i-1 &= n^2 \\
\therefore \sum_{i=1}^n \log i &= \log n!
\end{aligned}$$

Selection Sort running time.

```

for i = 0 to n-1
    min = i
    for j = i+1 to n-1
        if a[j] < a[min]
            min = j
    if min != i
        swap A[j] & A[min]
    
```

$$= \sum_{i=0}^{n-1} \left(\sum_{j=i+1}^{n-1} (1) + 2 \right)$$

$$= \sum_{i=0}^{n-1} \left((n-1 - (i+1) + 1) + 2 \right)$$

$$= \sum_{i=0}^{n-1} \left((n-1 - i - 1 + 1) + 2 \right)$$

$$= \sum_{i=0}^{n-1} (n - i - 1 + 2)$$

$$= \sum_{i=0}^{n-1} (n - i + 1)$$

$$= n(n-1-0+1) - \frac{(n-1)(n)}{2} + (n-1-0+1)$$

$$= n(n) - \frac{n(n-1)}{2} + n$$

$$= n \left[n - \frac{(n-1)}{2} + 1 \right]$$

$$= n \left[\frac{2n - (n-1) + 2}{2} \right]$$

$$= n \left[\frac{2n - n + 1 + 2}{2} \right] = n \frac{(n+3)}{2} = \frac{n^2}{2} + \frac{3n}{2}$$

$$= \frac{n^2}{2} + \frac{3n}{2} \in O(n^2)$$

Insertion Sort:

Insertionsort ($A[0 \dots n-1]$)

for $i = 2$ to $n-1$

key = $A[i]$ — (1)

$j = i-1$ — (1)

While $((key < A[j]) \&\& (j > 0))$

$\sum_{j=i-1}^{n-1} (2)$ $A[j+1] = A[j]$ — 1

$j = j-1$ — 1

$A[j+1] = key$ — 1

$\sum_{i=2}^{n-1}$

$$T(n) = \sum_{i=2}^{n-1} \left(\sum_{j=i-1}^{n-1} (2) + 3 \right)$$

$$= \sum_{i=2}^{n-1} (2(n-1-(i-1)+1) + 3)$$

$$= \sum_{i=2}^{n-1} (2(n-1-i+1+1) + 3)$$

$$= \sum_{i=2}^{n-1} (2(n-i+1) + 3)$$

$$= 2 \sum_{i=2}^{n-1} (n-i+1) + \sum_{i=2}^{n-1} (3)$$

$$= 2n(n-1-2+1) - \frac{(n-1)(n)}{2} + (n-1-2+1)$$

$$+ 3(n-1-2+1)$$

$$= 2n(n-2) - \frac{(n-1)n}{2} + (n-2) - 3(n-2)$$

$$= \frac{4n(n-2) - (n-1)n + 2(n-2) - 6(n-2)}{2}$$

$$= \frac{4n^2 - 8n - n + 1 + 2n - 4 - 6n + 12}{2}$$

$$= \frac{4n^2 - 12n + 8}{2} = 2(n^2 - 3n + 2)$$

$$= 2(n^2 - 3n + 2)$$

$$= 2n^2 - 6n + 2 \in O(n^2)$$